

Project Summary – Practice Microservices Project (Employee Management System)

I want to build this project from scratch as if I am working in a real product-based company. Please act as a **Senior Java Architect (30+ years experience) and Trainer**. Don't just give me code—teach me every concept, every decision, and every step like a real mentor.

Goal

Build a **practice microservices project** in **2 days** to understand the complete microservices ecosystem before starting a large production-level project.

Tech Stack

- Java 21
- Spring Boot 3.x
- Spring Cloud
- Maven
- Spring Data JPA
- Spring Data MongoDB
- Spring Validation
- Lombok
- OpenFeign
- Kafka
- MySQL
- MongoDB
- Config Server
- Eureka Server
- API Gateway
- Postman
- Docker (learn during the project, not before)

Project Name

Employee Management System

Microservices

Infrastructure Services

1. Config Server
2. Eureka Server (Service Registry)
3. API Gateway

Business Services

1. Employee Service
2. Department Service
3. Notification Service

Total: **6 Projects**

Database

MySQL

- Employee
- Department

MongoDB

- Notifications

Features

Employee Service

- Add Employee
- Update Employee
- Delete Employee
- Get Employee
- Department details using Feign Client

Department Service

- CRUD APIs

Notification Service

- Consume Kafka events
- Save notification in MongoDB

Kafka Flow

Employee Created ↓ Kafka Topic (`employee-created`) ↓ Notification Service ↓ MongoDB

REST Communication

Employee Service → Department Service (using OpenFeign)

Expected Response Example

```
{
  "employeeId": 1,
  "name": "Keshav",
  "email": "abc@gmail.com",
  "department": {
    "departmentId": 10,
    "departmentName": "IT"
  }
}
```

Common Folder Structure (for every microservice)

```
controller
service
repository
entity
dto
mapper
config
exception
client
producer
consumer
```

Learning Objectives

I want to learn:

- Project architecture
- Microservices design
- Config Server
- Eureka Server
- API Gateway
- OpenFeign
- Kafka Producer
- Kafka Consumer
- MySQL integration
- MongoDB integration
- DTO vs Entity
- Validation
- Global Exception Handling
- Service-to-Service Communication

- Real-world folder structure
- Best coding practices
- Git commits after every milestone
- Debugging and common production issues

Training Rules

- Teach me exactly like a senior developer mentoring a junior developer.
- Explain every dependency, annotation, configuration, class, and design decision.
- Before writing code, explain **why** we are doing it.
- After each phase, explain interview questions related to that topic.
- Follow real company workflow:
 - Requirement
 - Design
 - Coding
 - Testing
 - Debugging
 - Git Commit
- Never skip steps.
- Build everything from scratch.
- Assume I am working in a real product-based company.
- Don't provide complete code in one go. Build the project incrementally, one step at a time, explaining everything in detail.

Final Goal

After completing this practice project, I should be confident enough to build a production-level Order Management System with multiple microservices, Kafka, Docker, monitoring, and advanced Spring Cloud components.